

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN - ĐIỆN TỬ



BÁO CÁO DỰ ÁN

DỰ ÁN 10: KHÓA SỐ BẢO MẬT (DIGITAL SAFE LOCK) LƯU MẬT KHẨU VÀO SRAM

Sinh viên thực hiện:	Nguyễn Văn Dương – 20241713E Phạm Chí Dũng – 20200106
Đề tài:	Thiết kế hệ thống khóa số bảo mật trên FPGA
Nền tảng triển khai:	Kit DE2-150, FPGA Cyclone IV GX EP4CGX150DF31C7
Phần mềm:	Intel Quartus Prime 18.1 Lite Edition

Hà Nội, 07/2026

LỜI NÓI ĐẦU

Trong các hệ thống nhúng và hệ thống số, bài toán điều khiển truy cập bằng mật khẩu là một ví dụ điển hình để kết hợp nhiều khối chức năng: nhận tín hiệu từ phần cứng, chống dội phím, điều khiển theo máy trạng thái hữu hạn, giao tiếp bộ nhớ và hiển thị trạng thái cho người dùng. Dự án “Khóa số bảo mật” được xây dựng nhằm hiện thực một hệ thống khóa điện tử đơn giản nhưng có cấu trúc RTL rõ ràng, có khả năng lưu và thay đổi mật khẩu thông qua bộ nhớ.

Báo cáo này trình bày quá trình phân tích yêu cầu, thiết kế kiến trúc, xây dựng các module Verilog, kiểm thử bằng testbench và tổng hợp trên kit DE2-150. Ở phiên bản mã nguồn hiện tại, giao diện bộ nhớ giữa FSM và bộ lưu mật khẩu đã được đóng gói theo dạng đọc/ghi có tín hiệu sẵn sàng; phần hiện thực sử dụng module RAM nội bộ để mô phỏng hành vi lưu mật khẩu. Kiến trúc này vẫn giữ hướng mở để thay module RAM nội bộ bằng bộ điều khiển SRAM vật lý IS61WV51216 trên bo mạch.

LỜI CAM ĐOAN

Nhóm sinh viên cam đoan nội dung báo cáo phản ánh quá trình thiết kế, mô phỏng và tổng hợp dự án khóa số bảo mật do nhóm thực hiện. Các số liệu tổng hợp được trích từ kết quả build Quartus của project `digital_safe_lock_1`. Những hình ảnh thực tế phần cứng và ảnh chụp phần mềm Quartus sẽ được bổ sung sau theo kết quả thí nghiệm trực tiếp.

TÓM TẮT

Dự án thiết kế hệ thống khóa số bảo mật trên FPGA. Người dùng nhập mật khẩu bằng các switch, xác nhận bằng nút nhấn Enter, quan sát trạng thái khóa bằng LED đỏ/xanh, LED 7 đoạn và LCD. Mật khẩu mặc định được khởi tạo là 00. Khi nhập đúng mật khẩu, hệ thống chuyển sang trạng thái mở khóa; khi nhập sai, hệ thống báo lỗi trong một khoảng thời gian rồi quay lại trạng thái khóa. Khi đang mở khóa, người dùng có thể thay đổi mật khẩu bằng cách đặt giá trị mới trên SW[7:0] và nhấn nút đổi mật khẩu.

Thiết kế được tổ chức theo các module: `digital_safe_lock`, `button_debounce`, `lock_fsm`, `internal_ram`, `hex_display` và `lcd_controller`. Testbench tự kiểm tra các kịch bản chính: mở khóa bằng mật khẩu mặc định, đổi mật khẩu, từ chối mật khẩu sai, mở khóa bằng mật khẩu mới và kiểm tra hệ thống sau nhiều lần nhập sai liên tiếp. Kết quả Quartus cho thấy thiết kế tổng hợp thành công trên FPGA Cyclone IV GX, sử

dùng 349 logic elements, 187 thanh ghi và đạt timing dương ở các mô hình phân tích.

MỤC LỤC

LỜI NÓI ĐẦU	i
LỜI CAM ĐOAN	i
TÓM TẮT	i
DANH MỤC HÌNH VẼ	iv
DANH MỤC BẢNG BIỂU	v
1 TỔNG QUAN ĐỀ TÀI	1
1.1 Bối cảnh và mục tiêu	1
1.2 Yêu cầu chức năng	1
1.3 Nền tảng phần cứng	2
2 CƠ SỞ LÝ THUYẾT	3
2.1 FPGA và thiết kế RTL	3
2.2 Máy trạng thái hữu hạn	3
2.3 Bộ nhớ SRAM và giao tiếp đọc/ghi	3
3 THIẾT KẾ HỆ THỐNG	4
3.1 Sơ đồ khối tổng thể	4
3.2 Phân công tín hiệu vào ra	4
3.3 Module top-level	5
3.4 Module chống dội nút nhấn	5
3.5 Module điều khiển khóa	6
3.6 Module bộ nhớ	6
3.7 Module hiển thị	6

4	MÔ PHỎNG VÀ KIỂM THỬ	7
4.1	Chiến lược testbench	7
4.2	Các ca kiểm thử	7
4.3	Đánh giá kết quả mô phỏng	8
5	TỔNG HỢP VÀ TRIỂN KHAI TRÊN QUARTUS	8
5.1	Thiết lập project	8
5.2	Kết quả tài nguyên	9
5.3	Kết quả timing	9
5.4	Kiểm tra trên kit	10
6	ĐỊNH HƯỚNG MỞ RỘNG SANG SRAM VẬT LÝ	11
6.1	Lý do cần bộ điều khiển SRAM	11
6.2	Giao diện đề xuất	11
6.3	Quản lý bus hai chiều	12
7	KẾT LUẬN	12
	TÀI LIỆU THAM KHẢO	13
	PHỤ LỤC A: MÃ NGUỒN CHÍNH	14

DANH MỤC HÌNH VẼ

1.1	Ảnh kit DE2-150 và mô hình khóa thực tế	2
3.2	Sơ đồ khối chức năng của hệ thống khóa số	4
3.3	Sơ đồ chuyển trạng thái của module lock_fsm	6
4.4	Dạng sóng mô phỏng testbench	8
5.5	Ảnh project Quartus sau khi compile	9
5.6	Ảnh thực nghiệm trạng thái mở khóa	10
5.7	Ảnh thực nghiệm trạng thái báo lỗi	11
6.8	Hướng mở rộng từ RAM nội bộ sang SRAM vật lý	12

DANH MỤC BẢNG BIỂU

1.1	Yêu cầu chức năng của hệ thống	2
3.2	Các tín hiệu vào ra chính	5
4.3	Các ca kiểm thử chính	7
5.4	Tài nguyên FPGA sau tổng hợp và fitter	9

1 TỔNG QUAN ĐỀ TÀI

1.1 Bối cảnh và mục tiêu

Khóa số bảo mật là hệ thống điều khiển truy cập dựa trên việc so khớp mật khẩu do người dùng nhập với mật khẩu đã lưu trong bộ nhớ. Với nền tảng FPGA, hệ thống có thể được mô tả hoàn toàn bằng phần cứng số, chạy song song theo xung clock và phản hồi trực tiếp tới các ngoại vi trên kit thí nghiệm.

Mục tiêu chính của dự án gồm:

- Thiết kế hệ thống khóa số có khả năng nhập, kiểm tra và đổi mật khẩu.
- Xây dựng cấu trúc RTL theo hướng module hóa, dễ kiểm thử và dễ mở rộng sang SRAM vật lý.
- Hiện thực giao diện người dùng bằng SW[7:0], KEY[0], KEY[1], KEY[2], LED đỏ, LED xanh, LED 7 đoạn và LCD.
- Mô phỏng các kịch bản vận hành bằng testbench tự kiểm tra.
- Tổng hợp thiết kế trên kit DE2-150 bằng Quartus Prime.

1.2 Yêu cầu chức năng

Các yêu cầu chức năng của hệ thống được tóm tắt trong Bảng 1.1.

Bảng 1.1 Yêu cầu chức năng của hệ thống

Nhóm chức năng	Mô tả
Nhập mật khẩu	Người dùng đặt giá trị mật khẩu 8 bit trên SW[7:0].
Xác nhận	Nhấn KEY[1] để yêu cầu hệ thống đọc mật khẩu đã lưu và so sánh.
Mở khóa	Nếu mật khẩu đúng, bật LED xanh, tắt LED đỏ và hiển thị trạng thái OPn.
Báo lỗi	Nếu mật khẩu sai, bật LED đỏ, tắt LED xanh và hiển thị Err trong một khoảng thời gian.
Đổi mật khẩu	Khi đang mở khóa, đặt mật khẩu mới trên switch và nhấn KEY[2]; hệ thống ghi mật khẩu mới vào bộ nhớ và hiển thị Chg.
Khóa lại	Khi đang mở khóa, nhấn Enter để quay lại trạng thái khóa.
Reset	KEY[0] reset hệ thống về trạng thái khóa và khởi tạo mật khẩu mặc định.

1.3 Nền tảng phần cứng

Project được cấu hình cho kit DE2-150 sử dụng FPGA Cyclone IV GX EP4CGX150DF31C7. Các ngoại vi được dùng gồm switch nhập dữ liệu, nút nhấn, LED đơn, LED 7 đoạn và LCD 16x2. Theo định hướng đề tài, mật khẩu cần được lưu vào SRAM vật lý IS61WV51216; trong mã nguồn hiện tại, khối `internal_ram` đóng vai trò bộ nhớ lưu mật khẩu với giao diện đọc/ghi tương tự để phục vụ kiểm thử chức năng và tổng hợp.

Chèn hình sau

Ảnh thực tế kit, dây nối, LED và trạng thái khóa sẽ được bổ sung sau.

`images/de2_150_real.jpg`

Hình 1.1 Ảnh kit DE2-150 và mô hình khóa thực tế

2 CƠ SỞ LÝ THUYẾT

2.1 FPGA và thiết kế RTL

FPGA là vi mạch logic khả trình, cho phép hiện thực mạch số bằng mô tả phần cứng. Thiết kế RTL mô tả dòng dữ liệu giữa các thanh ghi theo cạnh clock. Với hệ thống khóa số, cách tiếp cận RTL phù hợp vì các thao tác đều có bản chất tuần tự: chờ người dùng nhập, đọc bộ nhớ, so sánh dữ liệu, cập nhật trạng thái và điều khiển hiển thị.

Thiết kế trong project sử dụng một clock hệ thống 50 MHz. Các tín hiệu bất đồng bộ từ nút nhấn được đồng bộ và lọc đội trước khi đưa vào FSM. Việc dùng chung một miền clock giúp giảm rủi ro race condition và làm cho phân tích timing trong Quartus rõ ràng hơn.

2.2 Máy trạng thái hữu hạn

Máy trạng thái hữu hạn (FSM) là mô hình điều khiển gồm tập trạng thái hữu hạn, tập đầu vào, tập đầu ra và luật chuyển trạng thái. Trong khóa số, FSM đảm bảo các thao tác bảo mật diễn ra theo trình tự xác định. Ví dụ, người dùng chỉ có quyền đổi mật khẩu khi hệ thống đang ở trạng thái mở khóa.

Các trạng thái chính trong module `lock_fsm` gồm:

- `S_INIT_WR`: ghi mật khẩu mặc định vào bộ nhớ sau reset.
- `S_INIT_WAIT`: chờ bộ nhớ hoàn tất ghi khởi tạo.
- `S_IDLE`: khóa đang đóng, chờ Enter.
- `S_READ_CHECK`: đọc mật khẩu đã lưu và so sánh.
- `S_UNLOCKED`: khóa mở, cho phép khóa lại hoặc đổi mật khẩu.
- `S_ERR`: báo sai mật khẩu.
- `S_WRITE_CHG`: ghi mật khẩu mới.
- `S_CHG_DONE`: báo đổi mật khẩu thành công.

2.3 Bộ nhớ SRAM và giao tiếp đọc/ghi

SRAM bất đồng bộ ngoài chip thường có bus địa chỉ, bus dữ liệu hai chiều và các tín hiệu điều khiển active-low như `CE_N`, `OE_N`, `WE_N`. Khi đọc, FPGA đưa địa chỉ và nhả

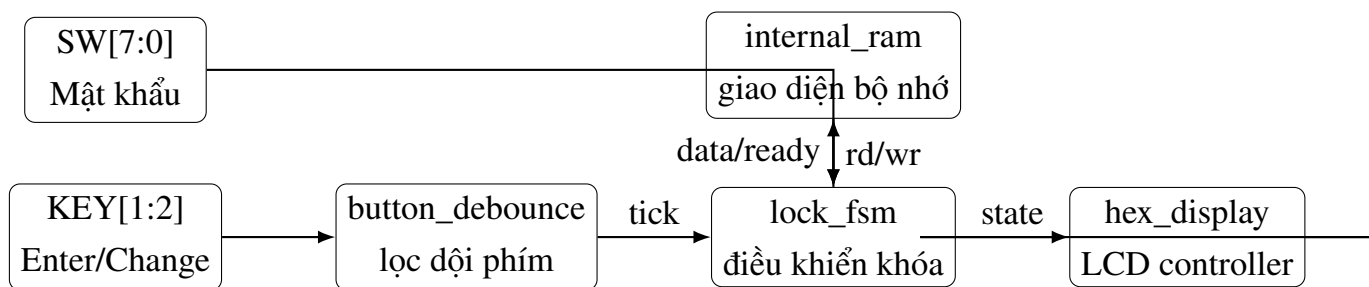
bus dữ liệu ở trạng thái trở kháng cao để SRAM lái dữ liệu. Khi ghi, FPGA lái dữ liệu lên bus và tạo xung ghi theo yêu cầu timing của chip.

Trong kiến trúc mục tiêu, module `sram_controller.v` sẽ che giấu các chi tiết timing này và cung cấp giao diện đơn giản cho FSM gồm `rd_en`, `wr_en`, `addr`, `data_in`, `data_out` và `ready`. Phiên bản hiện tại dùng `internal_ram.v` với cùng tinh thần bất tay: FSM phát yêu cầu, bộ nhớ trả `ready` khi thao tác hoàn tất. Nhờ vậy phần điều khiển khóa không phụ thuộc trực tiếp vào cách bộ nhớ được hiện thực.

3 THIẾT KẾ HỆ THỐNG

3.1 Sơ đồ khối tổng thể

Hình 3.2 biểu diễn kiến trúc chức năng của hệ thống. Các nút nhấn đi qua bộ chống dội trước khi tạo xung một chu kỳ cho FSM. FSM điều khiển đọc/ghi bộ nhớ, LED và mã trạng thái hiển thị. Khối LED 7 đoạn và LCD chuyển mã trạng thái thành thông tin người dùng có thể quan sát.



Hình 3.2 Sơ đồ khối chức năng của hệ thống khóa số

3.2 Phân công tín hiệu vào ra

Bảng 3.2 tóm tắt các tín hiệu chính. Trong project hiện tại, bus SRAM vật lý chưa được đưa ra top-level; do đó phần hiện thực thực tế đang dùng RAM nội bộ và các chân FPGA chủ yếu phục vụ giao diện người dùng.

Bảng 3.2 Các tín hiệu vào ra chính

Tín hiệu	Hướng	Chức năng
CLOCK_50	Input	Clock hệ thống 50 MHz.
KEY[0]	Input	Reset active-low.
KEY[1]	Input	Nút Enter, xác nhận mật khẩu hoặc khóa lại.
KEY[2]	Input	Nút đổi mật khẩu khi đang mở khóa.
SW[7:0]	Input	Giá trị mật khẩu 8 bit do người dùng nhập.
LEDR[0]	Output	LED đỏ báo khóa đóng hoặc lỗi.
LEDG[0]	Output	LED xanh báo khóa mở.
HEX2:HEX0	Output	Hiển thị --, OPn, Err, Chg.
LCD_*	Output	Giao tiếp LCD 16x2, hiển thị trạng thái dạng chữ.

3.3 Module top-level

Module `digital_safe_lock` là lớp kết nối giữa các ngoại vi vật lý và các khối RTL bên trong. Top-level nhận clock, switch, nút nhấn; sau đó đưa tín hiệu nút qua hai instance `button_debounce`. Các xung `enter_tick` và `change_tick` được đưa vào `lock_fsm`. FSM trao đổi với `internal_ram` bằng giao thức yêu cầu/sẵn sàng và xuất mã trạng thái cho LED 7 đoạn, LCD.

Hai tham số quan trọng của top-level là `DB_DELAY` và `TIMER_CYCLES`. Khi chạy phần cứng, `DB_DELAY=1_000_000` tương ứng khoảng 20 ms ở 50 MHz, còn `TIMER_CYCLES=100_000` tương ứng khoảng 2 giây. Khi mô phỏng, testbench giảm hai tham số này để mô phỏng nhanh hơn.

3.4 Module chống dội nút nhấn

Nút nhấn cơ khí thường sinh ra nhiều xung rất ngắn khi tiếp điểm đóng/mở. Nếu đưa trực tiếp vào FSM, một lần nhấn có thể bị hiểu thành nhiều lần nhấn. Module `button_debounce` xử lý vấn đề này bằng hai bước:

- Đồng bộ tín hiệu nút vào miền clock bằng hai flip-flop nối tiếp.
- Chỉ cập nhật trạng thái nút sau khi tín hiệu ổn định đủ số chu kỳ `DELAY_CYCLES`.

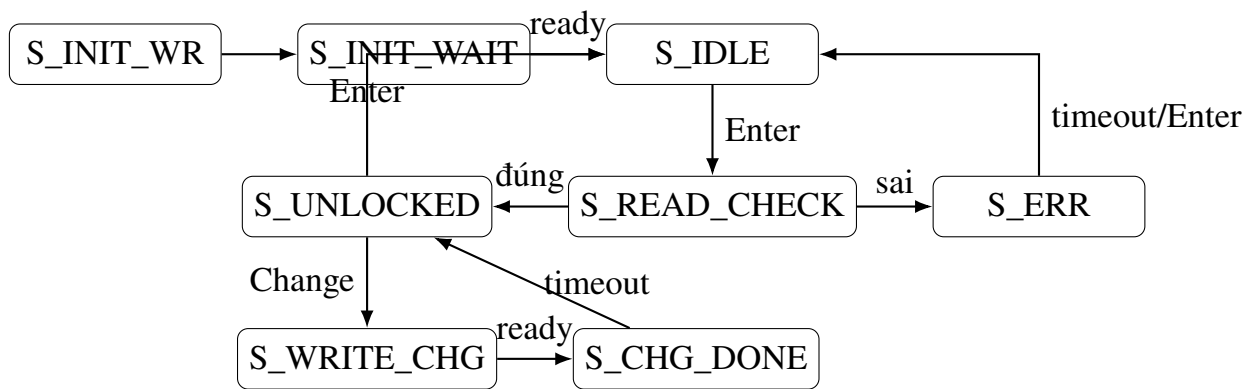
Đầu ra `o_btn_tick` chỉ lên mức 1 trong đúng một chu kỳ clock tại cạnh nhấn active-low. Đây là dạng tín hiệu phù hợp cho FSM vì tránh việc giữ nút quá lâu gây lặp

thao tác.

3.5 Module điều khiển khóa

Module `lock_fsm` là trung tâm điều khiển. Sau reset, FSM ghi mật khẩu mặc định 16'h0000 vào địa chỉ 0. Khi ở trạng thái khóa, nếu người dùng nhấn Enter, FSM đọc bộ nhớ tại địa chỉ 0 và so sánh 8 bit thấp với `i_sw`. Nếu khớp, hệ thống mở khóa; nếu không, hệ thống báo lỗi.

Khi đang mở khóa, `i_change_tick` làm FSM ghi mật khẩu mới {8'h00, `i_sw`} vào bộ nhớ. Sau khi nhận `i_sram_ready`, FSM hiển thị Chg. Nút Enter trong trạng thái mở khóa được dùng như lệnh khóa lại.



Hình 3.3 Sơ đồ chuyển trạng thái của module `lock_fsm`

3.6 Module bộ nhớ

Module `internal_ram` lưu một word 16 bit đại diện cho mật khẩu. Dù chỉ cần một word tại địa chỉ 0, module vẫn giữ các cổng `i_addr`, `i_rd_en`, `i_wr_en`, `i_data`, `o_data`, `o_ready`. Cách đóng gói này làm cho FSM có giao diện gần với bộ điều khiển SRAM hơn so với việc dùng trực tiếp một thanh ghi trong FSM.

Chu kỳ bộ nhớ gồm ba trạng thái nội bộ: IDLE, WAIT, DONE. Khi có yêu cầu đọc hoặc ghi, bộ nhớ xử lý dữ liệu và sau một chu kỳ phát `o_ready`. Trạng thái DONE giữ `ready` cho đến khi FSM hạ tín hiệu yêu cầu, tránh nhận nhầm cùng một yêu cầu nhiều lần.

3.7 Module hiển thị

Module `hex_display` giải mã `o_display_state` thành mẫu LED 7 đoạn active-low. Bốn mã trạng thái được dùng:

- 0: hiển thị --, trạng thái chờ/khóa.
- 1: hiển thị OPn, khóa mở.
- 2: hiển thị Err, sai mật khẩu.
- 3: hiển thị Chg, đổi mật khẩu thành công.

Module `lcd_controller` bổ sung kênh hiển thị LCD 16x2. Dòng đầu hiển thị tên hệ thống, dòng thứ hai hiển thị trạng thái như STATUS: LOCKED, STATUS: OPEN, WRONG PASSWORD! hoặc NEW PASS SAVED!.

4 MÔ PHỎNG VÀ KIỂM THỬ

4.1 Chiến lược testbench

Testbench `tb_digital_safe_lock.v` mô phỏng tương tác người dùng bằng cách gán giá trị cho switch và tạo xung nhấn nút trên KEY. Để mô phỏng nhanh, testbench override `DB_DELAY=1` và `TIMER_CYCLES=20`. Các tác vụ `reset_system`, `enter_password`, `wait_for_display_timer` và `check_result` giúp kích bản kiểm thử ngắn gọn, có khả năng tự báo Pass/Fail.

4.2 Các ca kiểm thử

Bảng 4.3 Các ca kiểm thử chính

Test	Thao tác	Kết quả mong đợi
1	Reset, nhập 00, nhấn Enter	Mở khóa, LEDG[0]=1, LEDR[0]=0.
2	Khi đang mở, đặt A5, nhấn Change	Ghi mật khẩu mới, hiển thị Chg.
3	Khóa lại, nhập sai 11, nhấn Enter	Báo lỗi, LEDG[0]=0, LEDR[0]=1.
4	Nhập mật khẩu mới A5, nhấn Enter	Mở khóa bằng mật khẩu mới.
5	Nhập sai nhiều lần liên tiếp	FSM không treo, bộ nhớ không bị sai lệch.
6	Sau nhiều lần sai, nhập lại A5	Hệ thống vẫn mở khóa đúng.

Chèn hình sau

Ảnh waveform hoặc transcript mô phỏng sẽ được bổ sung sau.

`images/modelsim_waveform.png`

Hình 4.4 Dạng sóng mô phỏng testbench

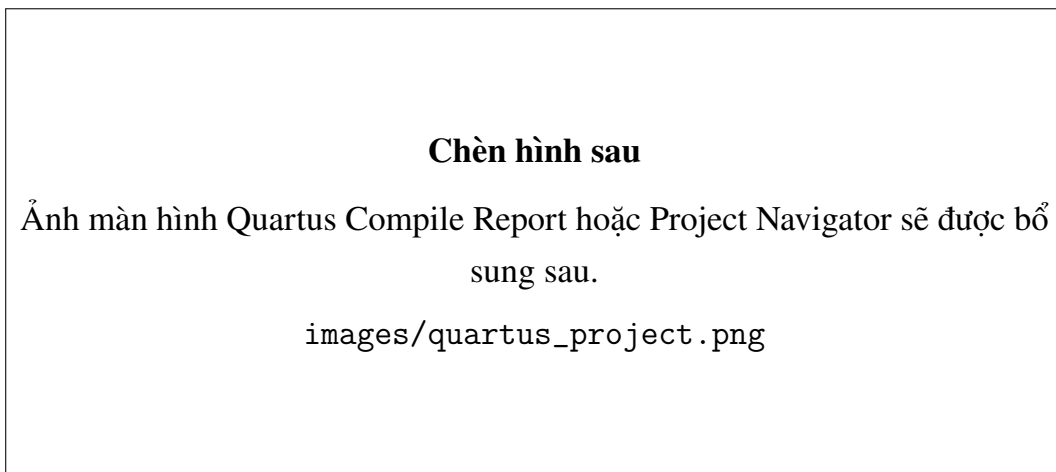
4.3 Đánh giá kết quả mô phỏng

Các ca kiểm thử bao phủ luồng vận hành chính và một số tình huống biên thực tế. Test 1 xác nhận quá trình khởi tạo mật khẩu mặc định. Test 2 và Test 4 xác nhận khả năng ghi và đọc lại mật khẩu mới. Test 3, Test 5 kiểm tra nhánh lỗi và khả năng trở về trạng thái khóa sau khi báo lỗi. Do testbench dùng cơ chế tự kiểm tra LED, lỗi chức năng cơ bản sẽ làm mô phỏng dừng tại \$stop.

5 TỔNG HỢP VÀ TRIỂN KHAI TRÊN QUARTUS

5.1 Thiết lập project

Project Quartus có tên revision `digital_safe_lock_1`, top-level entity là `digital_safe_lo`. Thiết bị đích là Cyclone IV GX EP4CGX150DF31C7. Kết quả build được tạo bởi Quartus Prime 18.1.0 Build 625 Lite Edition.



Hình 5.5 Ảnh project Quartus sau khi compile

5.2 Kết quả tài nguyên

Theo file `digital_safe_lock_1.fit.summary`, quá trình Fitter thành công lúc 10:02:34 ngày 30/06/2026. Tài nguyên sử dụng được tóm tắt trong Bảng 5.4.

Bảng 5.4 Tài nguyên FPGA sau tổng hợp và fitter

Chỉ tiêu	Kết quả
Total logic elements	349 / 149760, nhỏ hơn 1%
Total combinational functions	336 / 149760, nhỏ hơn 1%
Dedicated logic registers	187 / 149760, nhỏ hơn 1%
Total registers	187
Total pins	83 / 508, 16%
Total memory bits	0 / 6635520, 0%
Embedded multiplier 9-bit elements	0 / 720, 0%
Total PLLs	0 / 8, 0%

Kết quả cho thấy thiết kế nhỏ so với dung lượng FPGA. Vì mật khẩu hiện chỉ lưu trong một thanh ghi 16 bit, Quartus tối ưu thành logic register nên mục Total memory bits bằng 0. Khi chuyển sang block RAM hoặc SRAM vật lý, số liệu tài nguyên sẽ thay đổi theo cách hiện thực bộ nhớ.

5.3 Kết quả timing

Timing Analyzer báo slack dương cho clock `CLOCK_50`. Một số kết quả chính:

- Slow 1200 mV 85°C setup slack: 13.507 ns.
- Slow 1200 mV 85°C hold slack: 0.393 ns.
- Slow 1200 mV 0°C setup slack: 14.030 ns.
- Fast 1200 mV 0°C setup slack: 16.714 ns.
- Fast 1200 mV 0°C hold slack: 0.175 ns.

Các slack đều dương, do đó thiết kế đáp ứng ràng buộc timing ở tần số 50 MHz trong các mô hình phân tích được Quartus báo cáo.

5.4 Kiểm tra trên kit

Quy trình kiểm tra phần cứng đề xuất:

1. Nạp file `digital_safe_lock_1.sof` xuống kit DE2-150.
2. Nhấn reset bằng `KEY[0]`.
3. Đặt `SW[7:0]=00`, nhấn `KEY[1]`, quan sát LED xanh và chữ 0Pn.
4. Đặt `SW[7:0]=A5`, nhấn `KEY[2]`, quan sát Chg.
5. Nhấn Enter để khóa lại; nhập sai 11 để kiểm tra Err; nhập đúng A5 để mở lại.



Hình 5.6 Ảnh thực nghiệm trạng thái mở khóa

Chèn hình sau

Ảnh LED đỏ/HEX/LCD khi mật khẩu sai sẽ được bổ sung sau.

`images/hardware_error.jpg`

Hình 5.7 Ảnh thực nghiệm trạng thái báo lỗi

6 ĐỊNH HƯỚNG MỞ RỘNG SANG SRAM VẬT LÝ

6.1 Lý do cần bộ điều khiển SRAM

SRAM vật lý IS61WV51216 trên DE2-150 có bus dữ liệu hai chiều và yêu cầu timing đọc/ghi riêng. Không nên để FSM điều khiển trực tiếp các chân SRAM_CE_N, SRAM_WE_N, SRAM_OE_N vì điều đó làm FSM phức tạp và dễ phát sinh xung điều khiển sai. Thay vào đó, nên dùng một module `sram_controller` độc lập.

6.2 Giao diện đề xuất

Giao diện với FSM có thể giữ như hiện tại:

- `i_rd_en`, `i_wr_en`: yêu cầu đọc/ghi.
- `i_addr[18:0]`: địa chỉ bộ nhớ.
- `i_data[15:0]`: dữ liệu ghi.
- `o_data[15:0]`: dữ liệu đọc.
- `o_ready`: báo hoàn tất chu kỳ.

Phía ngoài module sẽ nối tới các chân vật lý:

- SRAM_ADDR, SRAM_DQ.
- SRAM_CE_N, SRAM_OE_N, SRAM_WE_N.

- Các tín hiệu byte enable nếu dùng: SRAM_LB_N, SRAM_UB_N.



Hình 6.8 Hướng mở rộng từ RAM nội bộ sang SRAM vật lý

6.3 Quản lý bus hai chiều

Điểm quan trọng nhất khi dùng SRAM vật lý là điều khiển bus SRAM_DQ. Trong chu kỳ ghi, FPGA được phép lái dữ liệu lên bus. Trong chu kỳ đọc hoặc khi rỗi, FPGA phải đặt bus ở trạng thái high-Z để SRAM hoặc mạch ngoài không bị tranh chấp bus. Một mô tả thường dùng trong Verilog là:

```

1 assign SRAM_DQ = drive_dq ? write_data : 16'hzzzz;
2 assign read_data = SRAM_DQ;

```

Tín hiệu drive_dq chỉ được bật trong pha ghi. Đây là yêu cầu bắt buộc để tránh hỏng dữ liệu và tránh dòng ngắn mạch trên bus.

7 KẾT LUẬN

Dự án đã xây dựng được hệ thống khóa số bảo mật có cấu trúc RTL rõ ràng trên FPGA. Hệ thống cho phép nhập mật khẩu bằng switch, xác nhận bằng nút nhấn, mở khóa khi mật khẩu đúng, báo lỗi khi mật khẩu sai và đổi mật khẩu khi đang ở trạng thái mở khóa. Các module được tách theo chức năng: chống dò phím, FSM điều khiển, bộ nhớ, LED 7 đoạn và LCD.

Testbench đã kiểm tra các luồng chức năng chính và sử dụng cơ chế tự so sánh đầu ra LED. Project Quartus tổng hợp thành công trên Cyclone IV GX, sử dụng rất ít tài nguyên và đạt timing dương ở clock 50 MHz. Điểm cần hoàn thiện tiếp theo là thay internal_ram bằng sram_controller nối trực tiếp tới SRAM vật lý IS61WV51216, cập nhật top-level với các chân SRAM_*, sau đó bổ sung ảnh thực nghiệm phần cứng và ảnh phần mềm Quartus vào báo cáo.

TÀI LIỆU THAM KHẢO

1. Intel, *Quartus Prime Standard Edition Handbook*, Intel Corporation.
2. Terasic, *DE2-150 User Manual*, Terasic Technologies.
3. ISSI, *IS61WV51216 Asynchronous SRAM Datasheet*, Integrated Silicon Solution Inc.
4. Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, Prentice Hall.
5. Project source files: `design/digital_safe_lock.v`, `design/lock_fsm.v`, `design/inter`
`tb/tb_digital_safe_lock.v`.

PHỤ LỤC A: MÃ NGUỒN CHÍNH

A.1 Module FSM điều khiển khóa

```
1 module lock_fsm #(
2     parameter TIMER_CYCLES = 28'd100_000_000 // Number of clock cycles to
        wait for the 2-second display timer
3 )
4     input wire i_clk, // System clock
5     input wire i_rst_n, // Active-low asynchronous reset
6
7     // User interface
8     input wire [7:0] i_sw, // 8-bit input from switches (represents the
        entered password)
9     input wire i_enter_tick, // 1-cycle pulse indicating the "Enter"
        button was pressed
10    input wire i_change_tick, // 1-cycle pulse indicating the "Change
        Password" button was pressed
11
12    // SRAM controller interface (Communication with the sram_controller
        module)
13    output reg o_sram_rd_en, // Signal to request a read operation from
        SRAM
14    output reg o_sram_wr_en, // Signal to request a write operation to
        SRAM
15    output reg [18:0] o_sram_addr, // Address in SRAM to access (Always 0
        in this basic design)
16    output reg [15:0] o_sram_data_out, // Data to send TO the SRAM (for
        writing)
17    input wire [15:0] i_sram_data, // Data received FROM the SRAM (after
        reading)
18    input wire i_sram_ready, // Handshake signal indicating the SRAM
        controller has finished its operation
19
20    // Outputs to the physical board
21    output reg o_ledr, // Red LED (Indicates Locked or Error state)
22    output reg o_ledg, // Green LED (Indicates Unlocked state)
23    output reg [2:0] o_display_state // 3-bit code sent to the hex_display
        module to show status text
```

```

24                                     // 0: Blank ("---"), 1: OPN ("OPn"), 2:
                                     ERR ("Err"), 3: CHG ("Chg")
25 );
26
27 // State Encoding for the Finite State Machine
28 localparam S_INIT_WR = 4'd0; // State: Send initial default password
    to SRAM on boot
29 localparam S_INIT_WAIT = 4'd1; // State: Wait for SRAM to finish
    writing the initial password
30 localparam S_IDLE = 4'd2; // State: Safe is locked, waiting for user
    input
31 localparam S_READ_CHECK = 4'd3; // State: Reading password from SRAM
    and comparing it with user input
32 localparam S_UNLOCKED = 4'd4; // State: Safe is unlocked, waiting for
    "Lock" or "Change Password" command
33 localparam S_ERR = 4'd5; // State: Wrong password entered, display
    error for 2 seconds
34 localparam S_WRITE_CHG = 4'd6; // State: Sending new password to SRAM
    to overwrite the old one
35 localparam S_CHG_DONE = 4'd7; // State: New password saved, display
    success message for 2 seconds
36
37 reg [3:0] state; // Current FSM state
38
39 // --- Separate Timer Logic Block ---
40 // This block manages the 2-second delay without cluttering the main
    state machine.
41 reg [27:0] timer;
42
43 // The timer is enabled only when the system is in an error state or
    showing a successful change
44 wire timer_en = (state == S_ERR) || (state == S_CHG_DONE);
45
46 // Flag that becomes true when the timer reaches the target count
47 wire timer_done = (timer >= TIMER_CYCLES);
48
49 always @(posedge i_clk or negedge i_rst_n) begin
50     if (!i_rst_n) begin
51         timer <= 28'd0;
52     end else if (timer_en) begin

```

```

53         // If timer is enabled and hasn't finished, increment it
54         if (!timer_done) timer <= timer + 1'b1;
55     end else begin
56         // Reset the timer immediately if we are not in a state that
           requires waiting
57         timer <= 28'd0;
58     end
59 end
60
61 // --- Main Finite State Machine ---
62 always @(posedge i_clk or negedge i_rst_n) begin
63     if (!i_rst_n) begin
64         // Initialize system into a locked, safe state upon reset
65         state <= S_INIT_WR;
66         o_sram_rd_en <= 1'b0;
67         o_sram_wr_en <= 1'b0;
68         o_sram_addr <= 19'd0;
69         o_sram_data_out <= 16'd0;
70         o_ledr <= 1'b1; // Default to Red LED ON (Locked)
71         o_ledg <= 1'b0;
72         o_display_state <= 3'd0; // Default to Blank display
73     end else begin
74         case (state)
75             S_INIT_WR: begin
76                 // Trigger a write operation to set the default
           password (00) at address 0
77                 o_sram_wr_en <= 1'b1;
78                 o_sram_addr <= 19'd0;
79                 o_sram_data_out <= 16'h0000;
80                 state <= S_INIT_WAIT;
81             end
82
83             S_INIT_WAIT: begin
84                 // Wait until the SRAM controller confirms the write is
           complete
85                 if (i_sram_ready) begin
86                     o_sram_wr_en <= 1'b0; // De-assert write request
87                     state <= S_IDLE; // Move to IDLE (ready for user)
88                 end
89             end

```

```

90
91     S_IDLE: begin
92         // Safe is locked. Display is Idle ("---"), Red LED is
           ON
93         o_ledr <= 1'b1;
94         o_ledg <= 1'b0;
95         o_display_state <= 3'd0;
96
97         // Wait for the user to press the Enter button
98         if (i_enter_tick) begin
99             // Request the SRAM controller to fetch the saved
               password from memory
100             o_sram_rd_en <= 1'b1;
101             o_sram_addr <= 19'd0;
102             state <= S_READ_CHECK;
103         end
104     end
105
106     S_READ_CHECK: begin
107         // Wait until SRAM fetches the data
108         if (i_sram_ready) begin
109             // Compare the lowest 8 bits from memory against
               the physical switches
110             if (i_sram_data[7:0] == i_sw) begin
111                 // Passwords match -> Unlock the safe
112                 state <= S_UNLOCKED;
113             end else begin
114                 // Passwords mismatch -> Deny access, go to
                   Error state
115                 state <= S_ERR;
116             end
117             o_sram_rd_en <= 1'b0; // De-assert read request
118         end
119     end
120
121     S_UNLOCKED: begin
122         // Safe is unlocked. Display "OPN", Green LED is ON
123         o_ledr <= 1'b0;
124         o_ledg <= 1'b1;
125         o_display_state <= 3'd1;

```

```

126
127     if (i_change_tick) begin
128         // User pressed "Change". We will overwrite the old
129             password with the current switch values
130         o_sram_wr_en <= 1'b1;
131         o_sram_addr <= 19'd0;
132         o_sram_data_out <= {8'h00, i_sw}; // Pad 8-bit
133             switch value to 16-bit word
134         state <= S_WRITE_CHG;
135     end else if (i_enter_tick) begin
136         // User pressed "Enter" while unlocked -> Lock the
137             safe again
138         state <= S_IDLE;
139     end
140 end
141
142 S_ERR: begin
143     // Access Denied. Display "ERR", Red LED is ON
144     o_ledr <= 1'b1;
145     o_ledg <= 1'b0;
146     o_display_state <= 3'd2;
147
148     // Remain in this state until the 2-second timer
149         finishes, OR the user presses Enter again to bypass
150     if (timer_done || i_enter_tick) begin
151         state <= S_IDLE; // Return to locked state
152     end
153 end
154
155 S_WRITE_CHG: begin
156     // Wait for the SRAM controller to finish saving the
157         new password
158     if (i_sram_ready) begin
159         o_sram_wr_en <= 1'b0; // De-assert write request
160         state <= S_CHG_DONE;
161     end
162 end
163
164 S_CHG_DONE: begin

```

```

160         // Password successfully changed. Display "CHG", Green
           LED remains ON
161         o_ledr <= 1'b0;
162         o_ledg <= 1'b1;
163         o_display_state <= 3'd3;
164
165         // Remain in this state to display the message until
           the 2-second timer finishes
166         if (timer_done) begin
167             state <= S_UNLOCKED; // Automatically return to the
               Unlocked state
168         end
169     end
170
171     default: state <= S_INIT_WR; // Fallback to initialization
           for safety
172     endcase
173 end
174 end
175
176 endmodule

```

A.2 Module bộ nhớ nội bộ

```

1 module internal_ram (
2     input wire i_clk, // System clock (50MHz)
3     input wire i_rst_n, // Active-low asynchronous reset
4
5     // FSM Interface
6     input wire i_rd_en, // Read enable signal from FSM
7     input wire i_wr_en, // Write enable signal from FSM
8     input wire [18:0] i_addr, // Memory address to read/write
9     input wire [15:0] i_data, // Data to be written into RAM
10    output reg [15:0] o_data, // Data read from RAM
11    output reg o_ready // High when the read/write operation is complete
12 );
13
14    // We only need to store a single 16-bit word (the password at
       address 0)

```

```

15 // To simulate a larger RAM if needed, this could be an array: reg
    [15:0] mem [0:1023];
16 // But since the FSM only writes to address 0, a single register is
    highly optimized.
17 reg [15:0] memory;
18
19 // Simple state machine to mimic SRAM controller timing (1-cycle
    delay)
20 reg [1:0] state;
21 localparam IDLE = 2'd0;
22 localparam WAIT = 2'd1;
23 localparam DONE = 2'd2;
24
25 always @(posedge i_clk or negedge i_rst_n) begin
26     if (!i_rst_n) begin
27         state <= IDLE;
28         o_ready <= 1'b0;
29         o_data <= 16'd0;
30         memory <= 16'd0;
31     end else begin
32         case (state)
33             IDLE: begin
34                 o_ready <= 1'b0;
35                 if (i_wr_en) begin
36                     memory <= i_data; // Write data immediately
37                     state <= WAIT;
38                 end else if (i_rd_en) begin
39                     o_data <= memory; // Read data immediately
40                     state <= WAIT;
41                 end
42             end
43
44             WAIT: begin
45                 // Add 1 cycle delay to perfectly mimic the old
46                 // external SRAM controller handshake
47                 o_ready <= 1'b1;
48                 state <= DONE;
49             end
50
51             DONE: begin

```

```

51         // Wait until FSM drops the enable signal before
           returning to IDLE
52         if (!i_rd_en && !i_wr_en) begin
53             o_ready <= 1'b0;
54             state <= IDLE;
55         end
56     end
57
58     default: state <= IDLE;
59 endcase
60 end
61 end
62
63 endmodule

```

A.3 Trích đoạn testbench

```

1  'timescale 1ns/1ps // Set simulation timescale: 1 nanosecond time unit, 1
   picosecond precision
2
3  module tb_digital_safe_lock();
4
5      // --- Signal Declarations ---
6      // Registers are used for signals that we generate/drive inside the
       testbench (Inputs to the DUT)
7      reg CLOCK_50; // 50MHz Clock
8      reg [17:0] SW; // 18 toggle switches for password input
9      reg [3:0] KEY; // 4 push buttons (Active Low)
10
11     // Wires are used to observe signals coming out of the DUT (Outputs
       from the DUT)
12     wire [17:0] LEDR; // Red LEDs
13     wire [8:0] LEDG; // Green LEDs
14     wire [6:0] HEX2; // 7-Segment Displays
15     wire [6:0] HEX1;
16     wire [6:0] HEX0;
17
18     // LCD physical interface wires
19     wire LCD_ON;
20     wire LCD_RS;

```

```

21  wire LCD_RW;
22  wire LCD_EN;
23  wire [7:0] LCD_DATA;
24
25  // --- Device Under Test (DUT) Instantiation ---
26  // Instantiate the Top Module with simulation-friendly parameters to
27  drastically speed up the test
28  // DB_DELAY = 1 ensures instant debouncing (no waiting millions of
29  cycles for a button press)
30  // TIMER_CYCLES = 20 ensures the 2-second display timer finishes in
31  just 400ns
32  digital_safe_lock #(
33      .DB_DELAY(20'd1),
34      .TIMER_CYCLES(28'd20)
35  ) dut (
36      .CLOCK_50(CLOCK_50),
37      .SW(SW),
38      .KEY(KEY),
39      .LEDR(LEDR),
40      .LEDG(LEDG),
41      .HEX2(HEX2),
42      .HEX1(HEX1),
43      .HEX0(HEX0),
44      .LCD_ON(LCD_ON),
45      .LCD_RS(LCD_RS),
46      .LCD_RW(LCD_RW),
47      .LCD_EN(LCD_EN),
48      .LCD_DATA(LCD_DATA)
49  );
50
51  // The memory is now internal to the FPGA (internal_ram), so we don't
52  need
53  // an external SRAM behavioral model anymore.
54
55  // --- Clock Generation ---
56  initial begin
57      CLOCK_50 = 0;
58      // Toggle the clock every 10ns, creating a 20ns period (50MHz
59      frequency)
60      forever #10 CLOCK_50 = ~CLOCK_50;

```

```

56     end
57
58     // --- Tasks for Modular, Self-Checking Tests ---
59
60     // Task 1: Perform a hardware reset
61     task reset_system;
62         begin
63             KEY[0] = 1'b0; // Assert Reset (Active Low)
64             #100 KEY[0] = 1'b1; // De-assert Reset after 100ns
65             #200; // Wait 200ns to allow the FSM to finish writing the
66                 default password to SRAM
67         end
68     endtask
69
70     // Task 2: Simulate a user entering a password and pressing a
71     specific button
72     // btn_type: 1 for Enter (KEY[1]), 2 for Change (KEY[2])
73     task enter_password(input [7:0] pass, input [1:0] btn_type);
74         begin
75             SW[7:0] = pass; // Flip the physical switches to set the
76                 password
77
78             if (btn_type == 1) begin
79                 KEY[1] = 1'b0; // Press the 'Enter' button
80                 #40; // Hold it down for 40ns (longer than 1 clock cycle to
81                     guarantee the debouncer catches it)
82                 KEY[1] = 1'b1; // Release the button
83             end else if (btn_type == 2) begin
84                 KEY[2] = 1'b0; // Press the 'Change Password' button
85                 #40;
86                 KEY[2] = 1'b1;
87             end
88
89             #200; // Wait 200ns for the SRAM operations and FSM processing
90                 to complete
91
92         end
93     endtask
94
95     // Task 3: Wait for the display timer (e.g., the 2-second "ERR" or "
96     CHG" display) to finish naturally

```

```

90 task wait_for_display_timer;
91     begin
92         // Since we parameterized TIMER_CYCLES to 20, the FSM will
93         // only wait 20 cycles (400ns).
94         // We wait 600ns here to ensure the timer has fully expired
95         // and the FSM has transitioned back to IDLE.
96         #600;
97     end
98 endtask
99
100 // Task 4: Self-checking result verifier. Automatically checks if the
101 // LEDs match our expectations.
102 task check_result(input exp_ledg, input exp_ledr, input [7:0] test_id)
103 ;
104     begin
105         if (LEDG[0] == exp_ledg && LEDR[0] == exp_ledr) begin
106             // If the outputs match expectations, print a Success
107             // message
108             $display("Pass: Test %0d (LEDG=%b, LEDR=%b)", test_id, LEDG
109                 [0], LEDR[0]);
110         end else begin
111             // If they don't match, print a Failure message with
112             // details and stop the simulation
113             $display("Fail: Test %0d - Expected LEDG=%b LEDR=%b, Got
114                 LEDG=%b LEDR=%b",
115                 test_id, exp_ledg, exp_ledr, LEDG[0], LEDR[0]);
116             $stop;
117         end
118     end
119 endtask
120
121 // --- Main Test Sequence (Execution Block) ---
122 initial begin
123     // Generate a waveform file (VCD) for viewing in tools like
124     // GTKWave
125     $dumpfile("waveform.vcd");
126     $dumpvars(0, tb_digital_safe_lock);
127
128     // Set initial, safe values for all inputs before resetting
129     SW = 18'h00000;

```

```

121 KEY = 4'b1111; // All buttons unpressed (Active Low)
122
123 $display("=====");
124 $display("DIGITAL_SAFE_LOCK-TESTBENCH_START");
125 $display("=====");
126
127 // --- Boot Sequence ---
128 reset_system();
129
130 // --- Test 1: Unlock with the default password ---
131 $display("\n[Test_1] Unlock with default password(00)");
132 enter_password(8'h00, 1);
133 check_result(1'b1, 1'b0, 1); // We expect Green LED ON, Red LED
    OFF
134
135 // --- Test 2: Change the password ---
136 $display("\n[Test_2] Change password to A5");
137 enter_password(8'hA5, 2); // Enter 'A5' and press the 'Change'
    button
138 wait_for_display_timer(); // Wait for the "CHG" message to finish
    displaying
139 $display("Password changed to A5.");
140
141 // Relock the safe before proceeding
142 $display("\nLocking the safe...");
143 enter_password(8'hA5, 1); // Pressing 'Enter' while unlocked will
    lock it
144
145 // --- Test 3: Attempt to unlock with an incorrect password ---
146 $display("\n[Test_3] Try unlocking with WRONG password(11)");
147 enter_password(8'h11, 1);
148 check_result(1'b0, 1'b1, 3); // We expect Green LED OFF, Red LED
    ON (Error)
149 wait_for_display_timer(); // Wait for the 2-second "ERR" display
    to finish
150
151 // --- Test 4: Attempt to unlock with the NEW correct password ---
152 $display("\n[Test_4] Unlock with NEW password(A5)");
153 enter_password(8'hA5, 1);

```

```

154     check_result(1'b1, 1'b0, 4); // We expect Green LED ON, Red LED
        OFF
155
156     // Relock the safe before proceeding
157     $display("\nLocking the safe...");
158     enter_password(8'h00, 1); // Password switch values don't matter
        when locking, just pressing 'Enter' is enough
159
160     // --- Test 5: Rapid succession of incorrect passwords (Spamming)
        ---
161     $display("\n[Test_5] Spam wrong password multiple times");
162     enter_password(8'h99, 1);
163     check_result(1'b0, 1'b1, 51);
164     wait_for_display_timer();
165
166     enter_password(8'h88, 1);
167     check_result(1'b0, 1'b1, 52);
168     wait_for_display_timer();
169
170     // --- Test 6: Verify the system still works after being spammed
        with errors ---
171     $display("\n[Test_6] Unlock again with correct password (A5)");
172     enter_password(8'hA5, 1);
173     check_result(1'b1, 1'b0, 6);
174
175     // Simulation is finished
176     #500;
177     $display("=====");
178     $display("SIMULATION COMPLETE");
179     $display("=====");
180     $finish;
181 end
182
183 endmodule

```